PCIe TLP Investigation Report

GSoC 2026 — Project 2.3: PCIe TLP Communication Framework using P4
P4 Language Consortium

Eslam Ashraf Ahmed
Electronics and Communications Engineering — Benha University
March 2026

Chapter 1: What is PCIe

A high-speed interface called Peripheral Component Interconnect Express facilitates communication between peripheral devices such as the GPU, SSD, FPGA accelerator, etc., with the CPU in a serial fashion. The PCIe interface facilitates serial communication between the peripheral devices, unlike the other interfaces that facilitate parallel communication. Two differential pairs are present in each lane of the PCIe interface: transmit and receive.

PCIe communication follows a hierarchical topology consisting of three main components: the Root Complex, optional Repeaters, and PCIe Endpoints.

The **Root Complex** is the interface between the host CPU, system memory, and the PCIe bus. It acts as the single source through which all PCIe traffic passes — either integrated directly into the CPU die (as in modern Intel and AMD processors) or as a discrete chipset component. In the context of this project, the Root Complex is the host-side controller that initiates and receives PCIe transactions with the FPGA.

Repeaters are signal conditioning devices that maintain signal integrity across the physical PCIe link. They fall into two categories — retimers and redrivers — both used to compensate for signal loss over PCB traces at high speeds. At PCIe Gen 4 and Gen 5 speeds, repeaters are commonly required in production systems.

Endpoints are the PCIe end devices — GPUs, NVMe SSDs, network cards, or in this project's case, the **AMD Xilinx Alveo U50 FPGA**. The Alveo U50 operates as a PCIe Endpoint, receiving Transaction Layer Packets (TLPs) from the Root Complex.

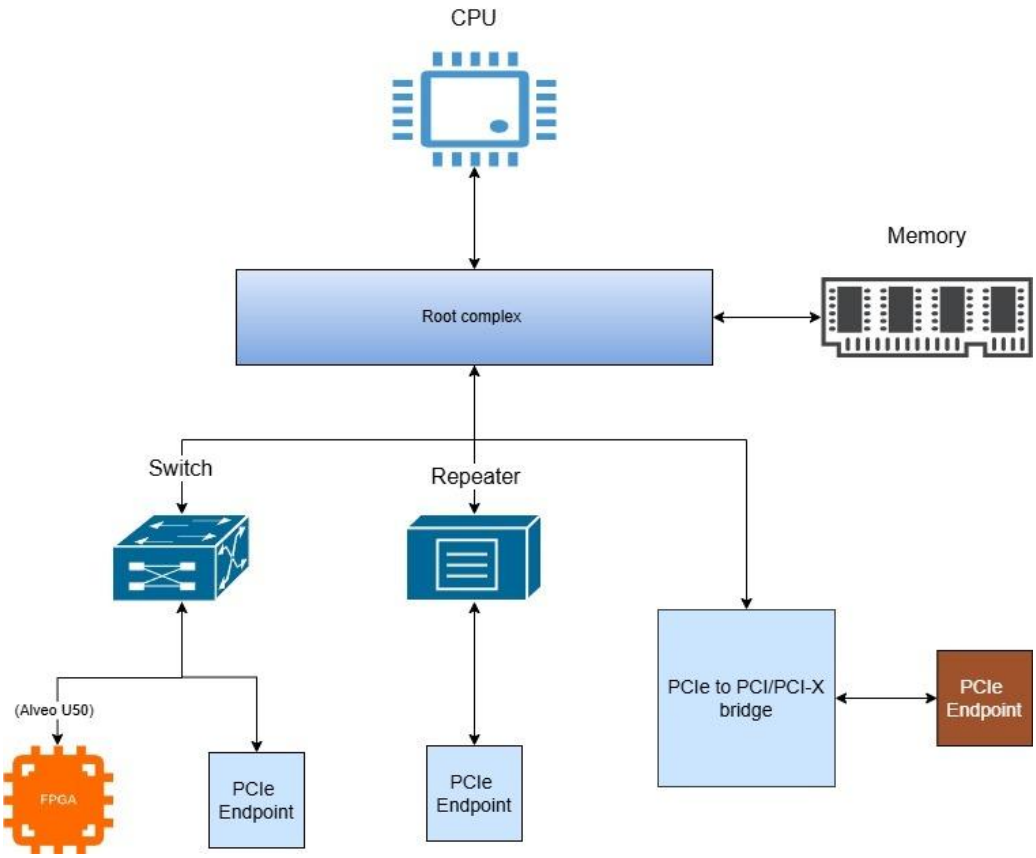

Figure 1 — PCIe Topology

Chapter 2: Transaction Layer Packets (TLPs)

TLP (Transaction Layer Packet) is the basic unit of data exchange in PCIe — it is the currency of communication between the host and the FPGA. Every single transaction happens through a TLP. Each TLP consists of two main parts: a header which contains the address, type, and size of the transaction, and an optional payload which carries the actual data.

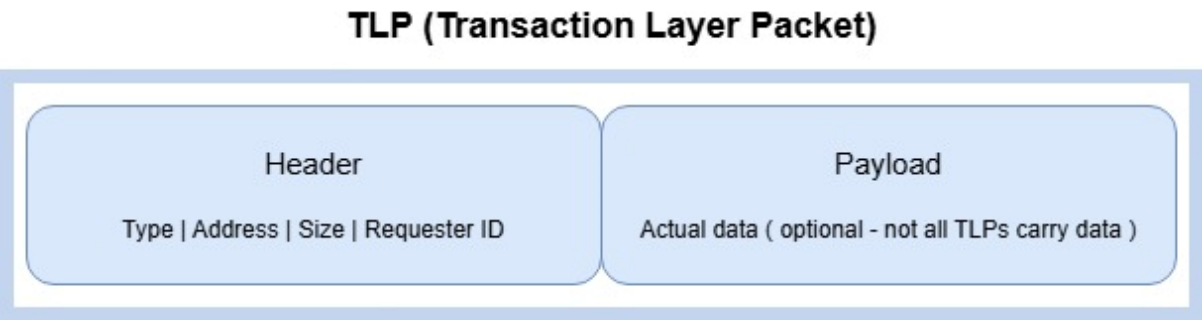

Figure 2 — TLP packet structure showing Header (always present) and Payload (optional)

TLP Types

There are four types of TLPs used in standard PCIe communication:

TLP Type	Sent By	Has Payload	Description
Memory Write Request	Host or FPGA	Yes	Sent when the host or FPGA wants to write data to the other side
Memory Read Request	Host or FPGA	No	Sent when the host or FPGA wants to read data from the other side
Completion with Data	Host or FPGA	Yes	Sent as a reply to a Memory Read Request, carrying the requested data
Completion without Data	Host or FPGA	No	Sent to confirm that a request was received and completed, with no data to return

Table 1. TLP Types

Chapter 3: RQ/CQ/RC/CC Interfaces

The AMD Xilinx PCIe UltraScale+ IP exposes four AXI4-Stream interfaces to the FPGA user logic, each handling a different direction of TLP communication. All interfaces are described from the FPGA's perspective:

Interface	Full Name (PG213)	Direction	Description
RQ	Requester reQuest	FPGA → Host	Used when the FPGA sends a request (Memory Read/Write) to the host
CQ	Completer reQuest	Host → FPGA	Used when the host sends a request (Memory Read/Write) to the FPGA
RC	Requester Completion	Host → FPGA	Used when the host sends a completion reply to the FPGA's earlier request
CC	Completer Completion	FPGA → Host	Used when the FPGA sends a completion reply to the host's earlier request

Table 2. RQ/CQ/RC/CC Interfaces

Chapter 4: AXI4-Stream Interface

The Xilinx PCIe UltraScale+ IP uses AXI4-Stream protocol to transfer TLP packets between the PCIe core and the FPGA user logic. Each of the four interfaces (RQ/CQ/RC/CC) uses AXI Stream to carry TLP data beat by beat.

Data transfer occurs only when both TVALID and TREADY are asserted in the same clock cycle. If the user logic deasserts TREADY while TVALID is high, the core freezes and holds the data stable on the bus until TREADY is asserted again.

TLAST marks the final beat of each TLP packet, while TUSER carries sideband descriptor information about the TLP being transferred.

Key AXI Stream Signals

Signal	Description
TDATA	Carries the actual TLP data across the interface
TVALID	Asserted by the core when data on TDATA is valid and ready to transfer
TREADY	Asserted by user logic when it is ready to accept data
TLAST	Asserted on the last beat of a TLP packet to indicate end of packet
TKEEP	Indicates which Dwords in TDATA contain valid data (1 bit per Dword)
TUSER	Carries sideband descriptor and metadata about the TLP being transferred (161 bits on RC interface)

Table 3 Key AXI Stream Signals

Chapter 5: PCIe Configuration Space

The PCIe Configuration Space is a standardized block of registers associated with every PCIe device. It serves as the device's identity card and resource declaration — it is how the host operating system discovers what a device is, what it needs, and how to communicate with it. Every PCIe Endpoint (like the Alveo U50) exposes up to 4 KB of Configuration Space.

5.1 Configuration Space Header (Type 0 — Endpoint)

The first 64 bytes form the **Type 0 Configuration Header**, used by all Endpoints including the Alveo U50.

				Byte Offset
Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header Type	Latency Timer	Cache Line Size	0Ch
Base Address Registers				10h 14h 18h 1Ch 20h 24h
Cardbus CIS Pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM Base Address				30h
Reserved			Capabilities Pointer	34h
Reserved				38h
Mac_Lat	Min_Gnt	Interrupt Pin	Interrupt Line	3Ch

*Figure 3 — **PCI Config Space Header Type 0***

The most important fields are shown in the table 4 below.

Offset	Field	Description
0x00	Vendor ID	Identifies the manufacturer (0x10EE = AMD Xilinx)
0x00	Device ID	Identifies the specific device model
0x04	Command	Controls the device's ability to respond to memory/IO transactions
0x04	Status	Reports device status and error conditions
0x08	Class Code	3-byte code identifying device function (e.g. network card, GPU, accelerator)
0x08	Revision ID	Hardware revision of the device
0x0C	Header Type	Indicates Type 0 (Endpoint) vs Type 1 (Root Port / Switch)
0x10–0x24	BAR0–BAR5	Six Base Address Registers defining memory regions the device exposes to the host
0x3C	Interrupt Line / Pin	Configures legacy interrupt handling

Table 4 Type 0 Config Space

The Class Code field is particularly relevant to this project. In the Phase 1 demo, a custom Class Code is programmed into the PCIe UltraScale+ IP in Vivado. When Linux enumerates the FPGA at boot, it reads this Class Code and recognizes the device under a custom class — rather than as a standard NIC or GPU — demonstrating how PCIe device identity can be defined at the hardware configuration level.

The Base Address Registers (BARs) define how much memory-mapped address space the device requires. During enumeration, the operating system performs a sizing procedure for each BAR by writing all 1s, reading back the value, and determining the required size based on the masked bits. It then assigns a physical address range to each BAR.

After address assignment, CPU accesses targeting BAR-mapped regions are translated by the Root Complex into Memory Read and Memory Write TLPs. These requests are delivered to the FPGA over the CQ (Completer Request) interface of the PCIe IP.

In this design, register access requests targeting BAR-mapped regions are detected on the CQ interface and passed through a cq_to_axi4lite bridge. This bridge converts PCIe TLP transactions into AXI4-Lite operations, enabling interaction with the VNP4 control plane. This mechanism forms the control path of the system, complementing the P4-based data path used for TLP processing.

5.2 Type 1 Configuration Header (Root Port / Switch)

Devices that act as bridges or switches use a **Type 1 header** instead of Type 0. The Type 1 header replaces BAR2–BAR5 with routing fields that allow the switch to forward TLPs to the correct downstream port.

				Byte Offset
Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header Type	Primary Latency Timer	Cache Line Size	0Ch
Base Address Register 0				10h
Base Address Register 1				14h
Secondary Latency Timer	Subordinate Bus Number	Secondary Bus Number	Primary Bus Number	18h
Secondary Status		I/O Limit	I/O Base	1Ch
Memory Limit		Memory Base		20h
Prefetchable Memory Limit		Prefetchable Memory Base		24h
Prefetchable Base Upper 32 bits				28h
Prefetchable Limit Upper 32 bits				2Ch
I/O Limit Upper 16 bits		I/O Base Upper 16 Bits		30h
Reserved			Capabilities Pointer	34h
Expansion ROM Base Address				38h
Bridge Control		Interrupt Pin	Interrupt Line	3Ch

*Figure 4 — **PCI Config Space Header Type 1***

Offset	Field	Description
0x18	Primary Bus Number	Bus number of the upstream bus
0x18	Secondary Bus Number	Bus number of the directly attached downstream bus
0x18	Subordinate Bus Number	Highest bus number reachable downstream through this port
0x20	Memory Base / Limit	Address range of downstream devices this port forwards traffic for

Table 5 Type 1 Config Space

These fields are what allow a PCIe Switch to route TLPs autonomously. When a TLP arrives at the switch, it checks the destination bus number against its Secondary and Subordinate Bus Number fields to decide which downstream port to forward the packet through — with no CPU involvement per packet. This is directly relevant to the Phase 2 demo, where a PCIe Switch sits between two Alveo U50 FPGAs and route TLPs between them using P4-controlled routing decisions.

References

1. AMD Xilinx, PCIe UltraScale+ IP Product Guide (PG213), https://docs.amd.com/r/en-US/pg213-pcie4-ultrascale-plus

2. iHalt10, PCIe Subsystem for AMD Xilinx, https://github.com/iHalt10/pcie_subsystem

3. iHalt10, VNP4 Framework, https://github.com/iHalt10/vnp4_framework

4. P4 Language Consortium, GSoC 2026 Project 2.3, https://github.com/p4lang/gsoc/blob/main/2026/ideas_list.md